



Unit-03

Directives and Forms



Prof. Arjun V. Bala

Computer Engineering Department

Darshan Institute of Engineering & Technology, Rajkot

✉ arjun.bala@darshan.ac.in

☎ 9624822202





Outline

- ✓ Directives
- ✓ Observable
- ✓ Dependency Injection
- ✓ Forms
 - ✓ Template Driven Forms
 - ✓ Reactive Forms

Draft Copy



योग: कर्मसु कोशलम्

Darshan
UNIVERSITY

Directives

Section - 01

Draft Copy



योग: कर्मसु कोशलम्

Darshan
UNIVERSITY

Directives

- ▶ Directives are classes that add additional behaviour to elements in your Angular applications.
- ▶ Directives are markers in the Document Object Model(DOM).
- ▶ Directives can be used with any controller or HTML tag which will tell the compiler what exact operation or behaviour is expected.
- ▶ There are some directives present that are predefined but if a developer wants, they can create new directives (custom-directive).
- ▶ Types of Directives
 - ➔ Component Directives:
Components are directives with templates. They are the building blocks of Angular applications, encapsulating both the UI (User Interface) and the behaviour of a part of the application.
 - ➔ Attribute Directives :
Change the appearance or behaviour of an element, component, or another directive.
 - ➔ Structural Directives :
Change the DOM layout by adding and removing DOM elements.

Attribute directives

- ▶ Attribute directives listen to and modify the behaviour of other HTML elements, attributes, properties, and components.
- ▶ Attribute directives change the appearance or behavior of DOM elements or components. It is used just like a normal HTML attribute.
- ▶ However, the directive should be enclosed within square brackets [] to bind it to the element.
- ▶ The most common built-in attribute directives are as follows:

Attribute Directive	Description	Example
NgClass	Adds and removes a set of CSS classes.	<code>[ngStyle]='{'color': 'blue', 'font-size': '14px'}'</code>
NgStyle	Adds and removes a set of HTML styles.	<code>[ngClass] = "dark"</code>
NgModel	Adds two-way data binding to an HTML form element.	<code>[(ngModel)]= "user.username"</code>

Structural Directives

- ▶ Structural directives change the structure of DOM by adding or removing elements at runtime.
- ▶ This type of Angular Directive is used to show or hide elements based on user authentication, render dynamic lists from APIs, or create interfaces like tabs and accordions that change based on user actions.
- ▶ The structural directives are applied to elements by prefixing the directive name with an asterisk (*) symbol.
- ▶ To use these directives in your angular application, importing CommonModule is necessary as they are part of this module.
- ▶ The most common built-in structural directives are as follows:

Directive	Description
ngIf	This directive is used to display or hide data in your application. When the given condition becomes TRUE, it will display the data, otherwise, it will not.
ngFor	ngFor is used to repeat a portion of elements from the given list of items.
ngSwitch	It checks multiple conditions.

Example (Structural Directives)

something.component.html

```
1 <div [ngSwitch]="semester">
2   <span *ngSwitchCase="1">Semester 1</span>
3   <span *ngSwitchCase="2">Semester 2</span>
4   <span *ngSwitchCase="3">Semester 3</span>
5   <span *ngSwitchCase="4">Semester 4</span>
6   <span *ngSwitchCase="5">Semester 5</span>
7   <span *ngSwitchCase="6">Semester 6</span>
8   <span *ngSwitchCase="7">Semester 7</span>
9   <span *ngSwitchCase="8">Semester 8</span>
10  <span *ngSwitchDefault>Invalid Semester</span>
11 </div>
12
13 <h1 *ngIf="semester>4">You Are Senior</h1>
14
15 <ul>
16   <li *ngFor="let temp of spi">{{ temp }}</li>
17 </ul>
```

something.component.ts

```
1 export class SomethingComponent {
2   semester = 10;
3   spi = [2.5,5.8,4.1,3.6];
4 }
```

Draft Copy

Rendering Array of Object with ngFor directive

something.component.html

```
1 <table>
2   <tr *ngFor="let stu of students; let i=index">
3     <td>{{i}}</td>
4     <td>{{stu.name}}</td>
5     <td>{{stu.marks}}</td>
6     <td>{{stu.isFailed}}</td>
7   </tr>
8 </table>
```

something.component.ts

```
1 export class SomethingComponent {
2   students = [
3     {
4       name: "arjun",
5       marks: 23,
6       isFailed: true
7     },
8     {
9       name: "darshan",
10      marks: 85,
11      isFailed: false
12    },
13    {
14      name: "rajkot",
15      marks: 14,
16      isFailed: true
17    }
18  ]
19 }
```

Custom Directives

- ▶ A custom directive is a user-defined directive that allows developers to extend the functionality of HTML elements.
- ▶ The attribute and structural built-in directives (covered in previous two chapters) offers very basic and pre-defined functionalities.
- ▶ However, with custom directives, you can add specific behaviors to HTML elements based on project requirements, user interactions, or changes in data.
- ▶ Features and Uses of Angular Custom Directives:
 - ➔ The first and most basic feature of a custom directive is to modify the structure or appearance of DOM elements based on your specific logic.
 - ➔ It allows you to embed event listeners to the HTML elements so that their behavior can be changed in response to user interaction.
 - ➔ You can create a reusable and modular piece of code using custom directives.
 - ➔ The custom directives can also help us add various features to the user interface, such as tooltip, drag and drop, form validation, dynamic styles and much more.

Custom Directives (Example)

- ▶ To create a custom directive, run the following command in Angular CLI –
 - ➔ `ng generate directive <directive-name>`
 - ➔ The above command will create a new directive with the specified name and add the necessary files for it. Among these files, the most important one is TypeScript file. This file has .ts extension.
- ▶ Step 01: `ng generate directive customstyle`
- ▶ Step 02: Add below line to constructor in `customstyle.directive.ts`

```
constructor(el: ElementRef) {  
  el.nativeElement.style.fontSize = '24px';  
}
```
- ▶ Step 03: `<p appCustomstyle>This text is styled using custom style directive</p>`

Draft Copy

Observable

- ▶ Angular makes use of observables as an interface to handle a variety of common asynchronous operations.
- ▶ For example:
 - ➔ You can define custom events that send observable output data from a child to a parent component
 - ➔ The HTTP module uses observables to handle AJAX requests and responses
 - ➔ The Router and Forms modules use observables to listen for and respond to user-input events
- ▶ Angular's `HttpClient` returns observables from HTTP method calls.
- ▶ For instance, `http.get('/api')` returns an observable.
- ▶ This provides several advantages over promise-based HTTP APIs:
 - ➔ Observables do not mutate the server response (as can occur through chained `.then()` calls on promises). Instead, you can use a series of operators to transform values as needed.
 - ➔ HTTP requests are cancellable through the `unsubscribe()` method
 - ➔ Requests can be configured to get progress event updates
 - ➔ Failed requests can be retried easily

Draft Copy

Dependency Injection (DI)

- ▶ Dependencies are services or objects that a class needs to perform its function. Dependency injection, or DI, is a design pattern in which a class requests dependencies from external sources rather than creating them.
- ▶ Angular's DI framework provides dependencies to a class upon instantiation.
- ▶ We can use Angular DI to increase flexibility and modularity in your applications.
- ▶ Consider two classes, A and B. Let's assume that class A uses the objects of class B. Normally, in OOPs, an instance of class B is created so that class A can access the objects. Using DI, we move the creation and binding of the dependent objects outside of the class that depend on them.
- ▶ Advantages of Dependency Injection
 - ➔ Dependency Injection helps in Unit testing.
 - ➔ Boilerplate code is reduced, as initializing of dependencies is done by the injector component.
 - ➔ Extending the application becomes more manageable.
 - ➔ It helps to enable loose coupling, which is essential in application programming.

Forms

- ▶ Handling user input with forms is the cornerstone of many common applications.
- ▶ Applications use forms to enable users to log in, to update a profile, to enter sensitive information, and to perform many other data-entry tasks.
- ▶ Angular provides two different approaches to handling user input through forms:
 - Reactive
 - Template-Driven
- ▶ Both capture user input events from the view, validate the user input, create a form model and data model to update, and provide a way to track changes.
- ▶ In the following slides we have information to help you decide which type of form works best for your situation.
- ▶ It introduces the common building blocks used by both approaches.
- ▶ It also summarizes the key differences between the two approaches, and demonstrates those differences in the context of setup, data flow, and testing.

Template-driven forms

- ▶ Template-driven forms rely on directives in the template to create and manipulate the underlying object model.
- ▶ They are useful for adding a simple form to an app, such as an email list signup form.
- ▶ They're straightforward to add to an app, but they don't scale as well as reactive forms.
- ▶ If you have very basic form requirements and logic that can be managed solely in the template, template-driven forms could be a good fit.
- ▶ Template-driven forms use two-way data binding to update the data model in the component as changes are made in the template and vice versa.
- ▶ Template-driven forms rely on directives defined in the **FormsModule**.

Directive	Details
NgModel	To provide two way binding
NgForm	To create top level form group instance
NgModelGroup	To group the NgModel

Steps to create Template Driven Form

- ▶ Step 01: Build the basic form
 - ➔ Define a sample data model
 - ➔ Include required infrastructure such as the FormsModule

- ▶ Step 02: Bind form tag using ngForm

```
<form #studentForm="ngForm" >
```

- ▶ Step 03: Bind form controls using ngModel

```
<input type="text" name="middlename" [(ngModel)]="data.middlename" #middlename="ngModel" required />
```

- ▶ Step 04: Track input validity and control status
- ▶ Step 05: Handle the form submit event using ngSubmit

```
<form #studentForm="ngForm" (ngSubmit)="processData()">
```

Draft
copy

Track Control State

- ▶ Adding the NgModel directive to a control adds class names to the control that describe its state.
- ▶ These classes can be used to change a control's style based on its state.

States	Class if true	Class if false
The control has been visited.	ng-touched	ng-untouched
The control's value has changed.	ng-dirty	ng-pristine
The control's value is valid.	ng-valid	ng-invalid

Draft Copy

Example (Template Driven Form)

signup-form.component.ts

```
1 import { Component } from '@angular/core';
2 import { FormsModule } from '@angular/forms';
3
4 @Component({
5   selector: 'app-login-form',
6   imports: [FormsModule],
7   templateUrl: './login-form.component.html',
8   styleUrls: ['./login-form.component.css']
9 })
10 export class LoginFormComponent {
11   data = {
12     firstname: '',
13     middlename: '',
14     lastname: ''
15   }
16
17   processData(){
18     console.log(this.data);
19   }
20 }
```

signup-form.component.html

```
1 <form #studentForm="ngForm" (ngSubmit)="processData()">
2   <table align="center" width="50%">
3     <tr>
4       <td>Enter First name</td>
5       <td><input
6         type="text"
7         name="firstname"
8         [(ngModel)]= "data.firstname"
9         #firstname="ngModel"
10        required
11      />
12     </td>
13   </tr>
14     ...
15     ...
16     ...
17   <tr>
18     <td colspan="2" align="center">
19       <input type="submit"
20         [disabled]="studentForm.invalid" />
21     </td>
22   </tr>
23 </table>
24 </form>
```

Reactive Forms

- ▶ Reactive forms provide a model-driven approach to handling form inputs whose values change over time.
- ▶ We will learn how to create and update a basic form control, progress to using multiple controls in a group, validate form values, and create dynamic forms where we can add or remove controls at run time.
- ▶ Reactive forms use an explicit and immutable approach to managing the state of a form at a given point in time.
- ▶ Each change to the form state returns a new state, which maintains the integrity of the model between changes.
- ▶ Reactive forms are built around observable streams, where form inputs and values are provided as streams of input values, which can be accessed synchronously.

Steps to setup reactive form

- ▶ Step 01: Import ReactiveFormsModule
- ▶ Step 02: Create a form group
- ▶ Step 03: Bind form group and control
- ▶ Step 04: Apply Validations
- ▶ Step 05: Display Validation Errors
- ▶ Step 06: Handle Form Submission

```
import { ..., ReactiveFormsModule, ...} from '@angular/forms';
```

```
studentForm = new FormGroup({  
  name: new FormControl(''),  
  rollNo: new FormControl(''),  
  semester: new FormControl(''),  
  spi: new FormControl('')  
});
```

```
<form [formGroup]="studentForm" (ngSubmit)="addStudent()">  
  Name : <input type="text" formControlName="name"/> <br/>  
  RollNo : <input type="text" formControlName="rollNo" /> <br/>  
  ...  
  ...  
  <input type="submit" [disabled]="studentForm.invalid" />  
</form>
```

```
addStudent(){  
  console.log(this.studentForm.value);  
}
```

Example (Reactive Form)

student-form.component.ts

```
1 import { ..., ReactiveFormsModule, ... } from '@angular/forms';
2 @Component({
3   selector: 'app-student-form',
4   imports: [ReactiveFormsModule],
5   ...
6 })
7 export class StudentFormComponent {
8   studentForm = new FormGroup({
9     name: new FormControl('', [Validators.required]),
10    rollNo: new FormControl(''),
11    semester: new FormControl('', [Validators.required]),
12    spi: new FormControl('')
13  });
14  addStudent(){
15    //get data using this.studentForm.value
16  }
17 }
```

student-form.component.html

```
1 <form [formGroup]="studentForm" (ngSubmit)="addStudent()">
2   Name : <input type="text" formControlName="name"/> <br/>
3   RollNo : <input type="text" formControlName="rollNo" /> <br/>
4   ...
5   <input type="submit" [disabled]="studentForm.invalid" />
6 </form>
```

Draft Copy

Creating FormGroup and Adding Validation Rules

```
1  import { Component } from '@angular/core';
2  import { FormControl, FormGroup, ReactiveFormsModule, Validators } from '@angular/forms';
3
4  @Component({
5    selector: 'app-reactive',
6    imports: [ReactiveFormsModule],
7    templateUrl: './temp.component.html',
8    styleUrls: ['./temp.component.css']
9  })
10 export class ReactiveFormComponent {
11   userForm = new FormGroup({
12     username: new FormControl('', [Validators.required]),
13     email: new FormControl('', [Validators.required, Validators.email]),
14     password: new FormControl('', [
15       Validators.required,
16       Validators.minLength(3),
17       Validators.maxLength(8)]),
18   })
19 }
```

Draft Copy

Displaying Validation Errors

Draft Copy

Go to component

```
1 <form [formGroup]="userForm">
2   <input type="text" formControlName="username" />
3   <span *ngIf="userForm.get('username')?.invalid && userForm.get('username')?.touched">
4     Please Enter UserName
5   </span>
6   <input type="text" formControlName="email" />
7   <span *ngIf="userForm.get('email')?.invalid && userForm.get('email')?.touched">
8     Please Enter Valid Email
9   </span>
10  <input type="text" formControlName="password" />
11  <span *ngIf="userForm.get('password')?.invalid && userForm.get('password')?.touched">
12    Please Enter Password
13  </span>
14  <button>Submit</button>
15 </form>
```

Introduction to FormBuilder

- ▶ What is **FormBuilder**?
- ▶ What is **need of FormBuilder**?
- ▶ A **service** in Angular to simplify creating and managing forms.
- ▶ **Reduces boilerplate code** when initializing **FormGroup** and **FormControl**.
- ▶ **Key Features:**
 - ↳ **Simplifies form** creation.
 - ↳ Allows **dynamic form generation**.
- ▶ **Advantages:**
 - ↳ **Reduces repetitive code**.
 - ↳ Supports **dynamic forms**.
 - ↳ Simplifies **managing nested FormGroup**s.
 - ↳ Enhances **scalability** and **readability** of form-related code.

Draft Copy

Setting Up FormBuilder

► Import **FormBuilder** in your Component and **Inject FormBuilder**.

► **Example :**

Using Constructor
(Ang. <= V.17)

```
1 import { Component } from '@angular/core';
2 import { FormBuilder } from '@angular/forms';
3
4 @Component({
5   selector: 'app-reactive',
6   imports: [],
7   templateUrl: './temp.component.html',
8   styleUrls: ['./temp.component.css']
9 })
10 export class ReactiveFormComponent {
11   constructor(private _fb: FormBuilder) {}
12 }
```

Using inject
method
(Ang. >= V.18)

```
1 import { Component, inject } from '@angular/core';
2 import { FormBuilder } from '@angular/forms';
3
4 @Component({
5   selector: 'app-reactive',
6   imports: [],
7   templateUrl: './temp.component.html',
8   styleUrls: ['./temp.component.css']
9 })
10 export class ReactiveFormComponent {
11   _fb = inject(FormBuilder);
12 }
```

Creating FormGroup Using FormBuilder & Adding Validations

- ▶ Use **FormBuilder** and **Validators** to **initialize FormGroup with controls** and **apply Validations** respectively:

- ▶ **Example :**

```
1  import { Component, inject } from '@angular/core';
2  import { FormBuilder, FormGroup,
3    ReactiveFormsModule, Validators
4  } from '@angular/forms';
5
6  @Component({
7    selector: 'app-reactive',
8    imports: [ReactiveFormsModule],
9    templateUrl: './temp.component.html',
10   styleUrls: ['./temp.component.css']
11 })
12 export class ReactiveFormComponent {
13   _fb = inject(FormBuilder);
14   userForm: FormGroup = this._fb.group({
15     username: ['', [Validators.required, Validators.minLength(3)]],
16     email: ['', [Validators.required, Validators.email]]
17   });
18 }
```

Draft Copy

Template Binding

```
<form [formGroup]="userForm">
  <div class="container">
    <div class="row">
      <div class="col-3">
        <label class="form-label text-light">User Name</label>
        <input type="text" formControlName="username" class="form-control" />
        <span *ngIf="userForm.get('username')?.errors?.['required'] && userForm.get('username')?.touched"
          class="text-danger">
          Please Enter User Name
        </span>
        <span *ngIf="userForm.get('username')?.errors?.['minlength'] && userForm.get('username')?.touched"
          class="text-danger">
          User Name length should be more than 3
        </span>
      </div>
    </div>
  </div>
</form>
```

Dynamic Forms with FormBuilder

► Use **FormBuilder** to dynamically add or remove controls:

► **Example :**

```
12 export class ReactiveFormComponent {
13   _fb = inject(FormBuilder);
14
15   userForm:FormGroup = this._fb.group({
16     username: ['', [Validators.required, Validators.minLength(3)]],
17     email: ['', [Validators.required, Validators.email]]
18   });
19
20   addAgeControl() {
21     this.userForm.addControl('age',
22       this._fb.control('', [Validators.required,
23         Validators.min(18),
24         Validators.max(25)]));
25   }
26
27   removeAgeControl() {
28     this.userForm.removeControl('age');
29   }
30 }
```

Draft Copy

Template Binding with Dynamic form approach

```
33     <div class="row" *ngIf="userForm.contains('age')">
34         <div class="col-3">
35             <label class="form-label text-light">Age</label>
36             <input type="text" FormControlName="age" class="form-control" />
37             <span *ngIf="userForm.get('age')?.errors?.['required'] && userForm.get('age')?.touched"
38                 class="text-danger">
39                 Please Enter Age
40             </span>
41             <span *ngIf="userForm.get('age')?.errors?.['minlength'] && userForm.get('age')?.touched"
42                 class="text-danger">
43                 Age should be more than 18
44             </span>
45         </div>
46     </div>
47
48     <div class="row">
49         <div class="col-3">
50             <button (click)="addAgeControl()">Add Age Control</button>
51             <button (click)="removeAgeControl()">Remove Age Control</button>
52         </div>
53     </div>
54
55     <div class="row mt-2">
56         <div class="col-3">
57             <button class="btn btn-primary">Submit</button>
58         </div>
59     </div>
```

Draft Copy



***Thank
You***



Prof. Arjun V. Bala

Computer Engineering Department

Darshan Institute of Engineering & Technology, Rajkot

✉ arjun.bala@darshan.ac.in

☎ 9624822202

